PROJECT AND TEAM INFORMATION

Project Title

SmartC Compiler: Smart Compilation with ML-Driven Optimization

Student / Team Information

Team Name: Team #	CodeOptix CD-VI-T089
Team member 1 (Team Lead)	Garg Naivaidhya 230211316 Naivaidhyag@gmail.com
Team member 2	Jain, Aarjav: 23021982: aarjavjain2210@gmail.com

Team member 3	Aditya, 230211640 adityarawatar01@gmail.com
Team member 4	Joshi, Devank 23021542 23021542@geu.ac.in

PROPOSAL DESCRIPTION

Motivation

Our motivation behind this project is to create a simple solution to optimize code for developers where beginners can easily optimize or learn from mistakes they make while designing and writing basic code. Be it practicing or writing actual applications, our project would help them identify basic to complex bugs in their syntax. Our compiler would be laced with a state-of-the-art Machine Learning model which would identify any sort of bugs or problems with the user's C code and fix those mistakes based on past user experiences and data. This project will optimize the initial code written by the user and would correct it according to the latest syntax and libraries of the version of C language which is being used on that device. This would ultimately ease the life of many developers as they can save time as the code written would be optimized to the best of the systems and the users need where basic tasks like improving time and space complexity of the program would be handled by the compiler itself.

State of the Art / Current solution

- Contemporary compilation techniques employ well-established algorithmic approaches such as dead code elimination, constant folding, loop unrolling, and common subexpression elimination [1]. Compilers like GCC and LLVM utilize sophisticated optimization passes, but these remain fundamentally static—they apply the same rules uniformly across all programs without learning from specific coding contexts [2].
- Recent academic research has explored machine learning's potential in compiler optimization. Researchers have demonstrated that neural networks can predict which optimization passes will deliver the best performance for specific code segments [4]. Studies have shown promising results using reinforcement learning to dynamically select optimization strategies [9] .
- However, most implementations remain either research prototypes or highly specialized for specific hardware architectures [3]. Mainstream compiler toolchains haven't widely adopted machine learning as a first-class optimization mechanism [11]. A comprehensive framework that integrates adaptive intelligence while maintaining correctness guarantees remains largely unexplored in practical educational settings [5,6]. Our approach bridges this gap by creating an accessible system that combines classical compilation principles with modern machine learning capabilities.

Project Goals and Milestones

- Develop an intelligent C compilation engine that successfully executes lexical analysis, syntax parsing, semantic validation, and intermediate code generation.
- Implement a machine learning module capable of analyzing intermediate code representations and identifying optimization opportunities.
- Create a practical tool that student programmers can use to enhance their code quality and

performance.
- Validate the system through comprehensive benchmarking and comparative analysis .

Milestones
- Week 1 to Week 4:
  - Complete architecture design.
  - Build basic compiler components (lexer, parser).
  - Setting up the development environment and version control for the compiler.
- Week 5 to Week 8:
  - Finalising semantic analysis and intermediate representation.
  - Building initial dataset of C programs with optimization metrics and labels.
  - Complete training the machine learning model.
- Week 9 to Week 12:
  - Integrating the model with the compiler pipeline.
  - Conducting initial testing and optimization for compiler design.
  - Prepare technical documentation and live demo.
  - Preparing initial benchmarking report.
- Week 13 to Week 16:
  - Ensuring that the System is compiling 90%+ of test programs.
  - Testing the model if it is improving code optimization as per the benchmark test results.
  - Getting ready with the final documentation of the project.

Project Approach

Our approach embraces modularity and separation of concerns. The system consists of two primary subsystems that operate collaboratively yet remain independently functional.

Compiler Architecture Component:

The compilation process unfolds across five distinct phases:
- Tokenization Phase – Using Flex, we decompose source code into meaningful tokens, handling keyword recognition, operator detection, and literal identification [1].
- Syntax Analysis Phase – Bison/Yacc constructs abstract syntax trees based on C grammar rules, validating structural correctness [1].
- Type and Scope Validation– The semantic analyzer performs type compatibility checking, scope resolution, and declaration verification [2].
- Intermediate Representation– Three-address code generation creates an abstraction suitable for both machine learning analysis and optimization [2].

Intelligent Optimization Component:

This layer operates independently from deterministic compilation:
- Feature Extraction: Numerical features derived from control flow graphs, loop structures, variable usage patterns, and memory access sequences [7].
- Pattern Recognition: Supervised learning models trained on curated datasets identify inefficient code structures [6,8].
- Optimization Suggestion Engine:A decision model recommends applicable optimizations specific to detected patterns [9,10].

System Architecture

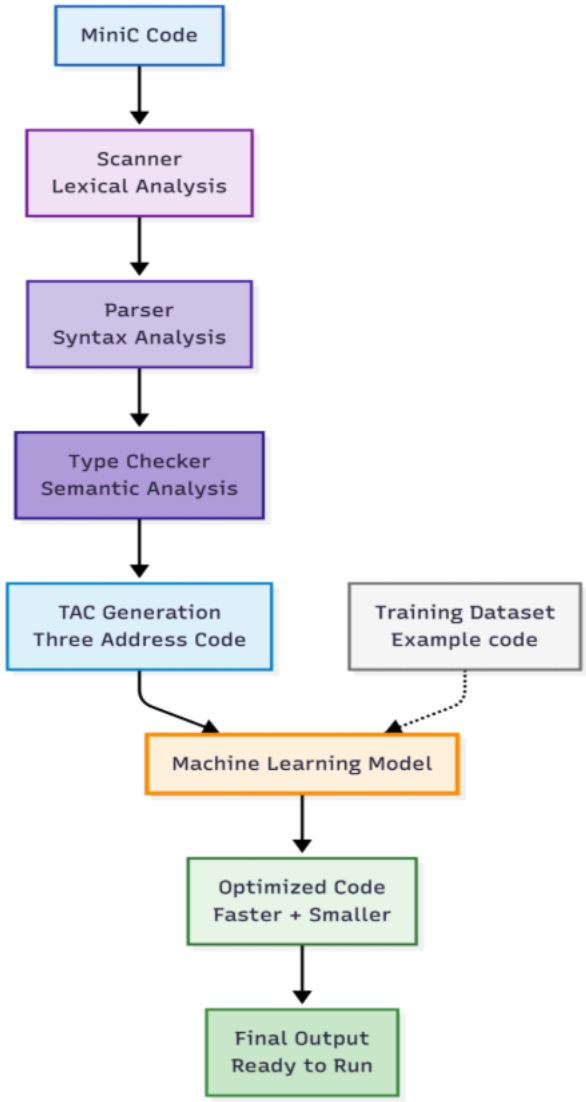

Figure 1.1 System architecture for SmartC Compiler

Project Outcome / Deliverables

By project completion, we will deliver the following tangible artefacts:

- Functional Compiler Implementation – A complete, working C compiler supporting core language features and demonstrating successful compilation of diverse C programs.
- Machine Learning Module – A trained model with associated preprocessing pipeline capable of analyzing intermediate code and generating optimization suggestions with quantifiable accuracy metrics.
- Performance Analysis Report– Detailed documentation comparing execution characteristics between traditionally compiled code and ML-guided optimization, including benchmarking

results on various algorithm types.

- **Technical Documentation:**
  1. Specifying system architecture including the component diagrams.
  2. Documenting api usage for compiler needs.
  3. Describing the features and dataset definitions.
  4. Tweaking the hyperparameters to improve the model training process.

- Live System Demonstration – A presentation showcasing the compiler processing real C programs, generating optimization suggestions, and illustrating performance improvements.

- GitHub Repository – Complete source code with comprehensive documentation, enabling future development and reproducibility.

- These deliverables collectively demonstrate the feasibility of integrating intelligent machine learning into traditional compilation workflows while maintaining system correctness and reliability.

Assumptions

- Dataset Availability: We're assuming we can find or create lots of C code examples that show both slow and fast versions, and that the patterns the AI learns from these examples will work when it sees new code it hasn't trained on.

- ML Applicability:We're assuming machine learning can genuinely spot opportunities to make code better, and that there's a connection between how code looks and how it can be optimized that the AI can figure out.

- Compiler Correctness: We're assuming any changes the AI suggests won't mess up what the program does or add new bugs - we'll double-check everything before applying optimizations.

- Resource Constraints: We're assuming we can build this within a normal school project timeline and with whatever computers we have access to (no supercomputers needed).

- Language Scope: We're focusing only on basic C features (not the whole complicated language) and assuming that's enough to prove our idea works.

- Tool Availability: We're assuming the tools we need (Flex for scanning, Bison for parsing, Python for AI) will work on our computers without major headaches.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson, 2006.[1]
2. Cooper, K., & Torczon, L. Engineering a Compiler (2nd ed.). Morgan Kaufmann, 2011.[2]
3. Ashouri, A. H., et al. A Survey on Compiler Autotuning using Machine Learning. ACM Computing Surveys, 2018. [3]
4. Wang, Z., & O'Boyle, M. Machine Learning in Compiler Optimization. IEEE Proceedings, 2018.[4]
5. Kovac, M., et al. Machine Learning-based Code Optimization Techniques. ACM Transactions, 2022.[5]
6. Cummins, C., et al. DeepTune: End-to-end Learning of Compiler Heuristics. PACT, 2017.[6]
7. Leather, H., et al. Automatic Feature Generation for Machine Learning-based Optimizing Compilation. CGO, 2009.[7]
8. Chen, T., et al. Learning to Optimize Tensor Programs. NeurIPS, 2018. [8]
9. Huang, Q., et al. Autotuning Compiler Optimization with Reinforcement Learning. CGO, 2019.[9]
10. Stephenson, M., et al. Meta Optimization: Improving Compiler Heuristics. PLDI, 2003.[10]
11. Agakov, F., et al. Using Machine Learning to Focus Iterative Optimization. CGO, 2006. [11]
12. Park, E., et al. Predictive Modeling in Compiler Optimization. IEEE Micro, 2012. [12]